Data Structure, Quiz 1 (2026) — Solution

A. Single Choice Questions (5% each, 60%)

(1) Which of the following statements about abstract data types, data structures, and classes is TRUE?

(A) An ADT specifies exactly how its data must be laid out in memory.

(B) Encapsulation means that a user must read the implementation of an ADT before using its operations.

(C) A class describes what the data look like (the state) and what the data can do (the behavior); a user of a class sees only this state and behavior, which is why classes are analogous to ADTs.

(D) The data structure is the logical view of the data, while the ADT is its physical implementation.

(C)

Key concepts: ADT, data structure, class, encapsulation.

- **(A) False.** An ADT is a logical description of the data and the allowed operations **without regard to how they will be implemented**.
- **(B) False.** Encapsulation **hides** the implementation details from the user's view.
- **(C) True.** This is the lecture's description of a class; a user of a class only sees the state and behavior of its objects, just as a user of an ADT only sees its operations.
- **(D) False.** It is reversed: the ADT is the logical view, and its implementation (the physical view) is the data structure.

Lecture source: deck 01 §1.5 (cells 21–23, 25–27), §1.8 (cells 34–35).

(2) In the logic-gate simulation from class, `LogicGate` is the parent class; `BinaryGate` and `UnaryGate` are derived from it, and `AndGate` is derived from `BinaryGate`. `LogicGate` declares the operation `performGateLogic` but does not implement it; each concrete gate implements it. A `Connector` object holds two gates, one at each end. Which statement is FALSE?

(A) An `AndGate` **Is-a** `LogicGate`.

(B) A `Connector` **Has-a** `LogicGate`; it is not part of the gate hierarchy.

(C) `LogicGate` is an abstract class, so no plain `LogicGate` object can be created; each concrete gate overrides `performGateLogic`.

(D) Because a `Connector` uses gates, `Connector` must be a subclass of `LogicGate`.

(D)

Key concepts: inheritance (Is-a), composition (Has-a), abstract class, overriding.

- **(A) True.** `AndGate` → `BinaryGate` → `LogicGate`: an `AndGate` is a `LogicGate`.
- **(B) True.** The lecture states that the `Connector` class does not reside in the gate hierarchy; a connector **has** two gates.
- **(C) True.** Because `LogicGate` does not know how each gate performs its logic, the operation is left unimplemented, making `LogicGate` an abstract class. Each gate overrides `performGateLogic`.

- **(D) False.** Using or holding objects of another class is a Has-a relationship, not inheritance.

Lecture source: deck 01 §1.13 (cells 228–232, 236, 239–240, 242, 244, 246, 253–254).

(3) Two algorithms solve the same problem. Their exact step counts are

$$T_P(n) = 100n + 5000, \qquad T_Q(n) = \frac{n^2}{2}.$$

Which statement is TRUE?

(A) P is $O(n)$ and Q is $O(n^2)$. Q needs fewer steps for small inputs such as $n = 100$, but P needs fewer steps once n is large enough.

(B) Because T_P has much larger constants, Q is faster than P for every input size.

(C) Both have the same dominant term n^2, because the constant 5000 is larger than n for small inputs.

(D) P is $O(100n)$, which is a different Big-O class from $O(n)$.

(A)

Key concepts: dominant term, crossover point.

- Dropping constants and lower-order terms: $T_P(n) = O(n)$ and $T_Q(n) = O(n^2)$.
- At $n = 100$: $T_P = 15000$ and $T_Q = 5000$, so Q is faster for this small input.
- Solving $100n + 5000 < n^2/2$ gives $n > 100 + \sqrt{20000} \approx 241.4$, so P takes fewer steps for every $n \geq 242$. Big-O describes this long-run behavior.

(B) ignores growth rate. **(C)** The dominant term of T_P is $100n$, not n^2; the size of a constant does not change the order of growth. **(D)** Big-O drops the coefficient: $O(100n) = O(n)$.

Lecture source: deck 02 §2.3 (cells 40–47, 54).

(4) Match the four anagram-detection solutions from class with their order of magnitude, in this order: (1) Checking Off, (2) Sort and Compare, (3) Brute Force, (4) Count and Compare. For Brute Force, count how many candidate strings it must generate; assume sorting costs $O(n \log n)$.

(A) $O(n)$, $O(n \log n)$, $O(2^n)$, $O(n)$

(B) $O(n^2)$, $O(n \log n)$, $O(n!)$, $O(n)$

(C) $O(n^2)$, $O(n)$, $O(n!)$, $O(n)$

(D) $O(n^2)$, $O(n \log n)$, $O(n!)$, $O(1)$

(B)

Key concept: anagram detection.

1. **Checking Off**: each of the n characters of `s1` scans the remaining characters of `s2`: $1 + 2 + \cdots + n = n(n+1)/2 = O(n^2)$.
2. **Sort and Compare**: the comparison loop is $O(n)$, but the two sorts cost $O(n \log n)$ and dominate.
3. **Brute Force**: generates all $n!$ arrangements of `s1` as candidates: $O(n!)$ candidates (even more work if building each string is counted).
4. **Count and Compare**: $T(n) = 2n + 26$, which is $O(n)$, using two lists of 26 counters (trading space for time).

(C) is the trap from the lecture: the loop after sorting is linear, but the sorting is not. **(D)** The two counting loops still depend on n.

Lecture source: deck 02 §2.4 (cells 65–66, 75–76, 82, 85–86, 93–95), §2.5–2.6 (cell 116).

(5) Four ways to build a dynamic array holding n integers were compared in class:

1. insert each new element at the **front** of the array;
2. append each new element at the **end**;
3. reserve capacity for n elements first, then append each element at the end;
4. create the array with n slots first, then assign each element by index.

Which statement is TRUE?

(A) Every single append in Method 2 takes $O(1)$ time in the worst case.

(B) Method 3 has a better Big-O than Method 2, because reserving capacity avoids reallocations.

(C) Method 1 takes $O(n^2)$ time in total, because every insertion at the front shifts all existing elements.

(D) Method 4 takes $O(n^2)$ time, because creating n slots already costs $O(n)$ for each slot.

(C)

Key concept: performance of dynamic-array operations.

- **(A) False.** Appending is **amortized** $O(1)$. An append that triggers a reallocation copies all elements and costs $O(n)$.
- **(B) False.** Methods 2, 3, 4 are all $O(n)$. Reserving capacity can make Method 3 faster in practice, but it does not change the Big-O.
- **(C) True.** Inserting at the front of an array holding k elements shifts k elements, so the total is $0 + 1 + \cdots + (n-1) = O(n^2)$.
- **(D) False.** Creating the n slots costs $O(n)$ in total, not per slot.

Lecture source: deck 02 §2.5–2.6 (cells 99, 102–105, 113, 116); deck 03 §A.1 (cells 22–23).

(6) Assume an integer uses 4 bytes and a memory reference (address) uses 8 bytes. A **compact array** stores 10 integers directly. A **referential array** stores 10 references, each referring to its own integer stored elsewhere in memory. Counting the integers themselves, how many bytes does each layout use in total?

(A) 40 bytes and 120 bytes

(B) 40 bytes and 80 bytes

(C) 40 bytes and 40 bytes

(D) 80 bytes and 120 bytes

(A)

Key concept: compact vs. referential arrays.

- Compact array: the array cells hold the integers themselves: $10 \times 4 = 40$ bytes.
- Referential array: the cells hold 10 addresses ($10 \times 8 = 80$ bytes) **plus** the 10 integers living elsewhere ($10 \times 4 = 40$ bytes): 120 bytes.

(B) forgets the integers that the references point to. A compact structure has no overhead for storing the sequence of memory references, which is why it uses less memory.

Lecture source: deck 03 §A.2 (cells 52–56, 59).

(7) Which of the following statements about storing a sparse matrix is TRUE?

(A) A matrix is called sparse when more than half of its entries are non-zero.

(B) The linear-list format stores the non-zero entries in an array sorted by their values.

(C) The Dictionary of Keys (DOK) format stores every entry of the matrix, including the zeros, as key-value pairs.

(D) The COOrdinate (COO) format stores each non-zero entry using three arrays: one for row indices, one for column indices, and one for values.

(D)

Key concepts: sparse matrix, COO, DOK, linear list.

- **(A) False.** A sparse matrix has a value of 0 for **most** elements; the lecture's rule of thumb is NNZ / size < 0.05.
- **(B) False.** The linear list stores each non-zero as a node `{row, col, value, next}`, linked in row-major order.
- **(C) False.** DOK maps each `(row, column)` key to its non-zero value; unspecified entries are assumed to be 0.
- **(D) True.** COO uses three subarrays: row indices, column indices, and values.

Lecture source: deck 03 §A.4 (cells 86–88, 91, 96, 114).

(8) Two empty lists are created with the List ADT: `u = UnorderedList()` (whose `add` inserts each new item the way the unordered list from class does) and `o = OrderedList()` (ascending order). Both lists then receive the same operations in this order:

`add(5)` `add(12)` `add(3)` `add(9)`
What are the contents of each list from the head to the last node?

(A) `u` : 5 12 3 9; `o` : 3 5 9 12

(B) `u` : 9 3 12 5; `o` : 12 9 5 3

(C) `u` : 9 3 12 5; `o` : 3 5 9 12

(D) `u` : 3 5 9 12; `o` : 3 5 9 12

(C)

Key concept: `add` in unordered vs. ordered linked lists.

- The unordered list adds each new node **at the head** ($O(1)$), so the last item added comes first: `9 3 12 5`.
- The ordered list traverses to the correct position to keep ascending order: `3 5 9 12`.

(A) would be the result of appending at the tail. **(B)** uses descending order. **(D)** assumes the unordered list also sorts its items.

Lecture source: deck 04 (cell 10), §4.5 (cells 41–44), §4.6 (cells 108–114), §4.6.1 (cell 122).

(9) The `remove(item)` operation of an unordered linked list traverses the list with two references: `current` marks the node being examined and `previous` follows one node behind it. The list has **at least two nodes**, and the item is found in the **first** node, so `previous` does not refer to any node. What must be changed to remove that node correctly?

(A) `head` must be changed to refer to the node after `current`, i.e. the second node.

(B) The link of `previous` must be changed to refer to the node after `current`.

(C) `head` must be set to refer to no node, because the list becomes empty.

(D) Nothing needs to change; the node will simply be skipped during later traversals.

(A)

Key concept: the head special case in `remove`.

Normally the link that changes is the `next` link of `previous`. When the first node is removed there is no previous node, so the pointer that must change is the **head** of the list: it now refers to the node after `current`.

(B) is impossible because `previous` refers to no node. **(C)** would lose every node after the first one. **(D)** The head would still refer to the removed node.

Lecture source: deck 04 §4.5 (cells 64–78).

(10) Which of the following statements about linked lists is FALSE?

(A) The nodes of a linked list do not have to be stored in contiguous memory; each node keeps a link to the next node.

(B) Accessing the item at position i of a singly linked list takes $O(1)$ time, just like indexing an array, because the head is known.

(C) In a doubly linked list each node keeps links to both the previous and the next node, which allows $O(1)$ insertions and deletions at arbitrary positions once the node is known.

(D) Adding an item at the head of an unordered linked list takes $O(1)$ time.

(B)

Key concepts: linked structures, traversal, doubly linked lists.

- **(A) True.** Relative positions are kept by following links, not by contiguous memory.
- **(B) False.** A linked list has no random-access indexing: reaching position i means following i links from the head, which is $O(n)$ in the worst case. Only arrays compute an address directly (`base + index * size`).
- **(C) True.** The `prev` and `next` links give a doubly linked list $O(1)$ update operations at arbitrary positions.
- **(D) True.** Head insertion only creates a node and relinks the head.

Lecture source: deck 04 §4.3 (cells 15–20), §4.6.1 (cell 122), STL linked containers (cell 126), A.1 doubly linked list (cells 141–142); deck 03 §A.1 (cells 15–16).

(11) A stack is implemented with a dynamic array in two different ways:

- **Version 1** keeps the top of the stack at the **end** of the array.
- **Version 2** keeps the top of the stack at the **beginning** (index 0) of the array.

Starting from an empty stack, a program pushes n items and then pops all n items. Which statement is TRUE?

(A) Both versions take $O(n)$ time in total, because both obey the LIFO rule.

(B) Both versions take $O(n^2)$ time in total, because a dynamic array shifts its elements on every operation.

(C) Version 1 takes $O(n^2)$ time in total, while Version 2 takes $O(n)$ time in total.

(D) Version 1 takes $O(n)$ time in total, while Version 2 takes $O(n^2)$ time in total.

(D)

Key concept: same ADT, different implementation cost.

- Version 1: adding or removing at the end of a dynamic array is amortized $O(1)$ / $O(1)$, so $2n$ operations cost $O(n)$.
- Version 2: every push or pop at index 0 shifts all current elements. With k items one operation costs $O(k)$, so the total is about $2(1 + 2 + \cdots + n) = O(n^2)$.

(A) Both versions are correct stacks, but identical ADT behavior does not imply identical cost. **(B)** Operations at the end of the array do not shift anything.

Lecture source: deck 05 §3.5 (cells 30, 36–42).

(12) The values 1, 2, 3, 4 are pushed onto an initially empty stack **in this order**. A `pop` may be performed at any time (between pushes or after them), and each popped value is printed. Which of the following output sequences is **impossible**?

(A) `1 2 3 4`

(B) `3 1 4 2`

(C) `2 4 3 1`

(D) `4 3 2 1`

(B)

Key concept: LIFO order.

- `1 2 3 4`: push 1, pop, push 2, pop, push 3, pop, push 4, pop.
- `3 1 4 2`: to print 3 first, 1 and 2 must already be on the stack with 2 on top of 1. After popping 3 the top is **2**, so 1 cannot be printed next. **Impossible.**
- `2 4 3 1`: push 1, push 2, pop (2), push 3, push 4, pop (4), pop (3), pop (1).
- `4 3 2 1`: push 1, 2, 3, 4, then pop four times.

Lecture source: deck 05 §3.3 (cells 10–11, 13–16), §3.4 (cells 21–23).

B. Short-answer questions, please provide the derivation for each question along with your answer (8% each, 40%)

(13) Briefly explain each of the following terms.

(a) Abstract data type (ADT)

(b) Column-major order

(c) Ordered list

(d) FIFO (first in, first out)

(a) Abstract data type (ADT). A logical description of a data type: what values it holds and what operations it supports, independent of how it is implemented. The user interacts only with the interface (for example, `push`, `pop`, `peek` of a Stack), so the implementation can change (vector-backed, linked) without affecting the user. This hiding is encapsulation.

(b) Column-major order. A way of laying out a two-dimensional array in one-dimensional memory in which an entire column is stored before the next column. Following the same idea as the row-major formula, for an R-row array `A[i][j]` is at `base + (j * R + i) * size`. (C++ two-dimensional arrays are row-major.)

(c) Ordered list. A list whose items are kept in an order determined by a property of the items (ascending or descending), so the items must support comparison. Its `add` must traverse to find the correct position, and `search` / `remove` can stop early once they pass the target.

(d) FIFO (first in, first out). The ordering principle of a queue: items are added at the rear and removed from the front, so the item that has been in the collection the longest is removed first. Examples: a checkout line, a cafeteria line, the queues an operating system uses to schedule processes, and the Hot Potato simulation.

Lecture source: deck 01 §1.5 (cells 21–23, 25–27); deck 03 §A.1 (cell 15), §A.3.1 (cells 76–79, 83); deck 04 §4.6–4.7 (cells 88–92, 99, 108–116); deck 05 §3.10 (cells 163–164, 168), §3.13 (cells 182–190).

(14-1) Evaluate the time complexity of the following code segment in terms of Big-O notation (assume no integer overflow):

```
int x = 0;
for (int i = 1; i <= n; i *= 2) {
    for (int j = 0; j < n; j++) {
        x++;
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < 100; j++) {
        x--;
    }
}

for (int k = 1; k < n * n; k *= 3) {
    x++;
}
```

(14-2) Evaluate the time complexity of the following code segment in terms of Big-O notation (assume no integer overflow):

```
x = 0
i = 1
while i <= n:
    for j in range(n):
        x += 1
    i *= 2

for i in range(n):
    for j in range(100):
        x -= 1
```

```
k = 1
while k < n * n:
    x += 1
    k *= 3
```

Answer: $O(n \log n)$

First segment. The outer loop runs for $i = 1, 2, 4, \ldots$ while $i \le n$, i.e. $\lfloor \log_2 n \rfloor + 1$ times. The inner loop always runs n times. Total: $n(\lfloor \log_2 n \rfloor + 1) = O(n \log n)$.

Second segment. $n \times 100 = 100n = O(n)$; the constant 100 is dropped.

Third segment. $k = 1, 3, 9, \ldots$ while $k < n^2$: about $\log_3(n^2) = 2 \log_3 n$ iterations, $O(\log n)$.

Overall: $O(n \log n) + O(n) + O(\log n) = \boxed{O(n \log n)}$.

> The first segment is a nested loop, but its dominant growth is $n \log n$ rather than quadratic: the outer loop doubles i, so it runs only $O(\log n)$ times.

Lecture source: deck 02 §2.3 (cells 33–47, 55–58) and the exercise in §2.4 (cell 96).

(15) Consider the conversion of the infix expression

`A * ( B + C * D ) - E / F`
to postfix form using a stack, as described in class.

(a) What is the resulting postfix expression?

(b) What is the maximum number of symbols (operators and left parentheses) in the operator stack at any one moment during the conversion?

(c) What is the equivalent prefix expression?

(a) Postfix: `A B C D * + * E F / -`

Token	Action	Operator stack (bottom → top)	Output so far
`A`	output		`A`
`*`	push	`*`	`A`
`(`	push	`* (`	`A`
`B`	output	`* (`	`A B`
`+`	push (top is `(`)	`* ( +`	`A B`
`C`	output	`* ( +`	`A B C`
`*`	`*` is higher than `+`: push	`* ( + *`	`A B C`
`D`	output	`* ( + *`	`A B C D`
`)`	pop until `(`: output `*`, `+`	`*`	`A B C D * +`
`-`	pop `*` (higher precedence), push `-`	`-`	`A B C D * + *`
`E`	output	`-`	`A B C D * + * E`
`/`	`/` is higher than `-`: push	`- /`	`A B C D * + * E`
`F`	output	`- /`	`A B C D * + * E F`
end	pop all: `/`, `-`		`A B C D * + * E F / -`

(b) Maximum: 4 symbols, `* ( + *`, right after the second `*` is pushed.

(c) Prefix: `- * A + B * C D / E F`

Fully parenthesized: `((A * (B + (C * D))) - (E / F))`. Moving each operator to its left parenthesis gives the prefix form.

Lecture source: deck 05 §3.9, §3.9.1, §3.9.2 (cells 100–131, 134–137, 139–141).

(16) Assume `q` is a queue that supports the Queue ADT operations (`enqueue`, `dequeue` returns the removed item, `isEmpty`, `size`) and `s` is a stack that supports the Stack ADT operations (`push`, `pop` returns the removed item, `peek`, `isEmpty`, `size`). Trace the following operations:

```
q = Queue()
s = Stack()
q.enqueue(4)
q.enqueue(9)
q.enqueue(2)
s.push(q.dequeue())
q.enqueue(s.peek() + 1)
s.push(q.dequeue())
s.push(7)
q.enqueue(s.pop())
```

(a) What are the final contents of `q` from front to rear?

(b) What are the final contents of `s` from bottom to top, and what does `s.peek()` return now?

(c) Which values were returned by the two `dequeue()` calls and by the `pop()` call?

Trace:

Operation	Queue `q` (front → rear)	Stack `s` (bottom → top)	Returned
`q.enqueue(4)`	`4`		
`q.enqueue(9)`	`4, 9`		
`q.enqueue(2)`	`4, 9, 2`		
`s.push(q.dequeue())`	`9, 2`	`4`	`dequeue()` → 4
`q.enqueue(s.peek() + 1)`	`9, 2, 5`	`4`	`peek()` → 4
`s.push(q.dequeue())`	`2, 5`	`4, 9`	`dequeue()` → 9
`s.push(7)`	`2, 5`	`4, 9, 7`	
`q.enqueue(s.pop())`	`2, 5, 7`	`4, 9`	`pop()` → 7

(a) `q` (front → rear): `2, 5, 7`.

(b) `s` (bottom → top): `4, 9`; `s.peek()` returns `9`.

(c) The two `dequeue()` calls returned `4` and `9`; `pop()` returned `7`.

Lecture source: deck 05 §3.4 (cells 20–23), §3.11 (cells 170–175), course Stack/Queue interface (cells 30–32, 178).

(17) A two-dimensional array `X` is stored contiguously in memory on a computer whose storage order (row-major or column-major) is not known in advance (zero-based indices; every index mentioned below is valid). It is known that

- `X[1][2]` is located at address **244**

- `X[3][0]` is located at address **212**
- `X[4][3]` is located at address **276**

(a) Is the storage order row-major or column-major? Justify your answer.

(b) What is the size of each element?

(c) How many rows does `X` have?

(d) What is the address of `X[0][0]`?

(e) What is the address of `X[2][4]`?

(a) Column-major.

Try **row-major** with C columns and element size S: $\text{addr}(X[i][j]) = b + (iC + j)S$.

$$b + (C + 2)S = 244, \qquad b + 3CS = 212.$$

Subtracting: $(C + 2 - 3C)S = (2 - 2C)S = 32$. Since $C \geq 1$, the left side is ≤ 0, which is impossible. So the array is **not** row-major.

Column-major with R rows: $\text{addr}(X[i][j]) = b + (jR + i)S$.

$$b + (2R + 1)S = 244 \quad (1), \qquad b + 3S = 212 \quad (2), \qquad b + (3R + 4)S = 276 \quad (3)$$

$(1) - (2)$: $(2R - 2)S = 32$. $(3) - (1)$: $(R + 3)S = 32$.

So $2R - 2 = R + 3$, giving $R = 5$ and $S = 32/8 = 4$.

(b) Element size $S = 4$.

(c) $R = 5$ rows.

(d) From (2): $b = 212 - 3 \times 4 = 200$.

(e) $\text{addr}(X[2][4]) = 200 + (4 \times 5 + 2) \times 4 = 200 + 88 = \boxed{288}$.

Check: $X[1][2] = 200 + 11 \times 4 = 244$, $X[3][0] = 200 + 3 \times 4 = 212$, $X[4][3] = 200 + 19 \times 4 = 276$

.

Lecture source: deck 03 §A.3.1 (cells 76–83).